

Left Behind, Not Forgotten: Reusing Regression Tests to Find Bugs in WebAssembly Runtimes

Ishrak Hayet, Mohsen EsfandiyariDoulabi, Giorgi Merabishvili, and Marcelo d’Amorim

Abstract—WebAssembly (WASM) is gaining traction as a portable, high-performance, and secure execution technology for its ability to deliver near-native speed across platforms. WASM runtimes are optimized systems capable of executing binary WASM modules. As complex software, WASM runtimes contain bugs. Fuzzing is a popular approach to find bugs in these systems, but a limit study we conducted involving three popular WebAssembly runtimes and existing fuzzers with existing seed corpora suggests that the continued bug-revealing ability of fuzzers is limited by the waning effectiveness of existing WASM seed corpora.

This paper reports on a large-scale study to assess the benefits of using regression tests to find bugs in WASM runtimes. The study explores two scenarios of use of regression tests: (1) *transplanting* such tests across runtimes and (2) using such tests as seed corpora for fuzzing. Our study revealed a total of 34 previously-unknown bugs across 3 WASM runtimes; 13 through transplantation and 21 through fuzzing. This number goes far beyond the number of bugs found through fuzzing using well-known seed corpora. Our findings yield both empirical observations and actionable recommendations.

Index Terms—WebAssembly, Fuzzing, Seed Corpora Evaluation, Differential Testing

I. INTRODUCTION

WebAssembly (WASM) is emerging as a portable, high-performance, and secure execution technology. Originally designed for web browsers, it has evolved into a general-purpose compilation target for languages such as C, C++, and Rust. WASM delivers near-native speed while providing strong security guarantees and consistent behavior across platforms. WASM runtimes (e.g., WASMTIME [1], WASMER [2], WAMR [3], and WASMEDGE [4]) are responsible for loading, verifying, and executing WASM binaries. These systems are highly optimized and widely deployed, powering applications that range from serverless and edge computing to embedded systems. However, like any complex software, WASM runtimes are prone to bugs. Their components such as interpreters, JIT compilers, and sandboxing layers can harbor subtle correctness, performance, and security issues. Such bugs may lead to crashes, miscompilations, or vulnerabilities. As WASM adoption grows, ensuring the robustness and correctness of its runtimes becomes critical. Developing effective methods to uncover and fix bugs in WASM runtimes is therefore an important research challenge.

Several fuzzers have been proposed to detect bugs in WASM runtimes, e.g., WADIFF [5], WAPPLIQUE [6], and WAS-MAKER [7]. These tools have previously uncovered many

inconsistencies, crashes, and specification deviations across runtime implementations. However, no prior work has studied the continued effectiveness of these fuzzers when used with the existing seed corpora. As observed across various domains [8], [9], [10], [11], [12], fuzzing effectiveness heavily depends on the quality and diversity of the input corpus. The quality and diversity of an outdated corpus may degrade over time as its effectiveness becomes saturated; thereby leading to limited coverage and fewer bugs found, whereas a well-curated corpus from a new source can substantially extend the effectiveness and relevance of existing fuzzers. Understanding how corpus quality influences WASM fuzzing outcomes remains a crucial step toward more systematic and effective testing of WASM runtimes.

With the goal of evaluating the performance of WASM fuzzers on three established seed corpora from the literature, we conducted a preliminary investigation involving three WASM runtimes and three fuzzers. Each fuzzing campaign was executed for 24 hours and repeated five times. The results revealed that the number of newly found bugs is substantially lower than that reported initially by these fuzzers on the same corpora. This pattern reflects the well-documented “saturation effect” in fuzzing [13], [14]: as targets mature, fixed seed corpora reveal fewer new bugs. A similar dynamic appears in compiler fuzzing, where bug discovery spikes after major version transitions and then declines [15]. These observations motivated us to examine alternative sources of seeds that might be more resilient to saturation.

Bugs in software systems tend to concentrate according to the Pareto distribution [16], which is why repeated bug fixes are usually spatially localized [17]. Intuitively, regression tests (i.e., that previously revealed bugs) are rich in features—they have high potential to reveal new bugs. Based on this conjecture, **this paper presents a large-scale empirical study on how regression tests can be used to reveal new bugs in WASM runtimes**. Specifically, we examine the effectiveness of two complementary approaches leveraging regression tests to find bugs in WASM runtimes: (1) *test transplantation*, where regression tests are executed across different runtimes, and (2) *fuzzing* with a seed corpus of regression tests. Although regression tests have been used before for test generation [18], [19], we are the first to systematically study the effectiveness of regression tests to find new bugs in the WebAssembly domain using both test transplantation and fuzzing.

To support our study, we introduce RTBENCH, a dataset of 584 WebAssembly regression tests collected from multiple runtimes that we use for transplantation and as seeds for fuzzing. Our results show that the two bug-finding mechanisms

Ishrak, Mohsen, Giorgi, and Marcelo are with North Carolina State University, Raleigh, NC 27695 USA. (e-mail: {ihayet, mesfand, gmerabi, mdamori}@ncsu.edu)

we employ are complementary in revealing bugs in WASM runtimes. Transplanting regression tests uncovered **13** previously unknown bugs which is a modest but significant number, when compared to the 10 bugs found by the dedicated fuzzer WAPPLIQUE after several hours of fuzzing. Using RTBENCH as a seed corpus markedly improved fuzzing effectiveness: fuzzers achieved higher coverage and exposed **21** additional bugs compared to three existing seed corpora used in the literature. Importantly, none of the bugs discovered through transplantation overlapped with those found by fuzzing, highlighting their complementary nature. The detected issues spanned compilation and runtime failures, including crashes and output mismatches. Notably, the most effective seeds originated from regression tests created two to three years ago, coinciding with the transition from WebAssembly version 1 to 2, while older tests, though less useful for fuzzing, continued to reveal unique issues through transplantation.

This paper makes the following contributions:

- ★ An in-depth empirical study of using regression tests from existing WASM runtimes to identify bugs through complementary test transplantation and fuzzing;
- ★ A benchmark study of three established WASM seed corpora from the literature as a baseline for comparison;
- ★ RTBENCH - a dataset of over five hundred real-world WebAssembly regression tests collected from the issue trackers and regression suites of major runtimes: Wasmtime [1], Wasmer [2], WAMR [3], and WasmEdge [4].
- ★ **34** previously-unknown bugs across 3 WASM runtimes. Our results are publicly available at https://github.com/anonymous4submission2026/WASM_Fuzzing_Study;

II. BACKGROUND

This section outlines background for the rest of the paper.

A. WebAssembly Module

WebAssembly (WASM) is a virtual instruction set architecture that is often used as a compilation target for various high-level languages. A WASM module is the fundamental unit of deployment and execution; it is a platform-independent binary containing functions, type definitions, and data. WebAssembly Text (WAT) format [20] is a textual representation of a WASM file. Decompilers from WASM to WAT exist [21], [22].

Figure 1 shows a recursive version of the factorial function implemented in C and its corresponding WAT code. The code snippet defines a standard recursive factorial routine which is compiled to an equivalent WASM module using the `clang` compiler. Figure 1(b) shows the different sections in the WAT format: the Type section defines type signatures (parameter and result types) on line 2; Code section contains the function bodies (locals and instructions) on line 3; and Export section assigns a public name ‘factorial’ to the function on line 14, so that it can be externally invoked. The WASM code section is of particular importance to fuzzing as it contains the function bodies that collectively determine module behavior. Each instruction in WebAssembly consists of an opcode accompanied by zero or more parameters and operands. Given that a WASM module is executed within a

stack-based virtual machine, instructions operate on an implicit operand stack rather than explicit registers.

The execution of each instruction modifies the stack state according to its defined semantics, ensuring that the side effects and resulting values conform to the instruction’s type signature and parameter specification.

```

1 // Factorial in C
2 long long factorial(long
  long n) {
3   if (n == 0) {
4     return 1;
5   } else {
6     return n * factorial(n
      - 1);
7   }
8 }

```

(a) C code.

B. WebAssembly Runtimes

A WebAssembly runtime is a stack-based virtual machine that provides an execution model to load, instantiate, and run programs that have been compiled as WASM binary modules [23]. A WASM runtime processes a module in three stages: (1) decoding, (2) validation, and (3) execution. Similar to a compiler, decoding converts the binary into an intermediate representation or directly into machine code. Validation ensures consistency of the types and the operand stack [24]. Execution takes place by directly invoking the `start` function, if the module defines one, or by loading the module –possibly from a different programming language or from the Command Line Interface (CLI)– and calling exported functions.

```

1 (module
2   (type (func (param i64) (
    result i64))) ;; Type
    section
3   (func (type 0) (param i64
    ) (result i64)
4     local.get 0
5     i64.eqz
6     if (result i64) (
7       local.get 0
8       local.get 0
9       i64.const 1
10      i64.sub
11      call 0
12      i64.mul
13    end)
14   (export "factorial" (func
    0)) ;; Export section
15 )

```

(b) Corresponding WAT code.

Fig. 1: Factorial example.

A WebAssembly runtime maintains three stacks: an operand stack to track instruction operands, a control stack to track control flow, and an initialization stack to track local variables of a function [25]. There are two ways to execute a WASM module: (1) by executing directly on standalone WebAssembly runtimes that run on an Operating System [23] and (2) by embedding a WASM module inside a host application (e.g. a browser) with the necessary glue code and programmatic API access to the WebAssembly runtimes. This paper focuses on the standalone versions of the WebAssembly runtimes where we directly use their CLI to execute WASM modules.

C. Mutational Fuzzing of WASM Runtimes

Several domain-specific fuzzers have recently been proposed to assess the robustness and security of WASM runtimes [26], [5], [6], [7]. Most of these fuzzers are *mutational* [5], [6], [7], i.e., they generate new inputs through systematic mutations of existing inputs as opposed to creating new inputs. Typically, a mutational fuzzer initializes a population with a *seed corpus* and iteratively applies mutation operators to individuals from the population. In WASM mutational fuzzing, mutation operators can be applied at a low- or high-level data representation. More precisely, mutation operators can either

modify WASM binary files or their corresponding abstract syntax trees (ASTs). For determining test outcomes, fuzzers typically rely on oracles that identify crashes or differential output among multiple runtimes.

As observed across multiple software domains, the quality and diversity of the seed corpus play a crucial role in determining the effectiveness of fuzzing [8], [9], [10], [11], [12].

III. WORKFLOW

Figure 2 illustrates the workflow that we use to find bugs in WASM runtimes. First, we extract (1) WASM test cases from GitHub issues of different runtime repositories and add them into RTBENCH (2). These mined test cases are then transplanted (3a) across runtimes (4) and also used separately as seeds for fuzzing (3b). All .wasm test cases (5) are then evaluated using differential testing (6) to identify behavioral inconsistencies across runtimes. Detected anomalies are filtered (6) to remove false positives and duplicates, then triaged (7) to produce bug reports (8).

A. RTBENCH

We propose RTBENCH, a seed corpus of WASM files mined from bug histories and regression tests. To construct our corpus, we collected seeds from real-world bug reports spanning a five-year period from 2020 to 2025. We mined the public GitHub issue trackers and regression testsuites of the four major WebAssembly runtimes, namely WASMTIME [1], WASMER [2], WAMR [3], and WASMEDGE [4] to collect 584 WASM files. The distribution of the collected seed testcases are shown in Table I.

TABLE I: Distribution of seeds in RTBENCH.

Runtime	Issues	Test suite	Total
WASMTIME	33	-	33
WASMER	15	274	289
WAMR	106	126	232
WASMEDGE	30	-	30
Total	184	400	584

We focus on issues that refer to either crashing inputs or the ones leading to differential output. For each candidate, we first identify the buggy runtime version from the issue report. We then execute the test case on that version to confirm bug reproducibility. Importantly, we also verify that none of these seeds reproduce crashes on the latest versions of the corresponding runtimes at the time of our study, confirming that the underlying bugs have been fixed. This approach provided us with a seed corpus grounded in actual historical bugs while ensuring they had indeed been buggy in the reported versions and that they no longer manifest in current releases. These checks help to form a realistic and reliable seed corpus for subsequent fuzzing and cross-runtime differential testing.

When collecting test cases from the runtime testsuites, we only select the .wasm files since the fuzzers expect only .wasm files as seeds (§II-C). For testcases collected from issues, we only keep the ones related to CLI bugs of the runtimes (§II-B). For testcases collected from both issues and testsuites, we filter out duplicates and non-deterministic cases.

B. Fuzzer Provenance Logging

For any large-scale fuzzing campaign, it is essential not only to generate diverse test cases but also to maintain a clear record of how those test cases were derived. As discussed at Section II-C, the two mutation-based fuzzers used in our study, WAPPLIQUE [6] and WASMAKER [7], generate new WASM modules by splicing one or more existing WASM modules. To ensure reproducibility and accountability across this process, we extended both fuzzers with a lightweight provenance logging mechanism that records the original seeds, the fragments used, and the mutation operations applied.

In WASMAKER [7], seed WASM binaries are parsed into per-function abstract syntax trees (ASTs), analyzed at the opcode level, and then mutated to form new inputs. We integrated provenance logging at the AST generation stage. Similarly, WAPPLIQUE [6] decomposes real WASM seeds into reusable code fragments from function bodies, which are then recombined and slightly mutated to synthesize new test cases. To ensure that these instrumentation changes did not alter fuzzer correctness or efficiency, we validated the modified fuzzers against their original behavior. Both fuzzers produced the same ratio of valid to invalid test cases as their uninstrumented versions. The overhead of provenance logging was negligible. This validation confirmed that our logging mechanism maintained functional equivalence while adding transparent, low-cost traceability to our fuzzing campaigns.

C. Differential testing

To systematically uncover inconsistencies across WebAssembly runtimes, we developed a large-scale differential testing framework that executes the same input WASM file under multiple runtime configurations and compares their outcomes. When running each input WASM file, we invoke all of the exported functions on different modes of the runtimes.

1) *Runtime Modes.*: Our differential testing framework supports a variety of execution and compilation modes across the tested runtimes. For WASMTIME, both direct interpretation and execution of precompiled (.cwasm) artifacts are tested. WASMER is evaluated under its Cranelift and LLVM backends. WAMR is exercised through its ahead-of-time compiler `wamrc`, native execution in AOT mode, interpreter, and JIT mode using the `iwasm` application. WASMEDGE is tested in JIT mode and interpreter mode with the `wasmedge` application, and compiled mode with `wasmedgec` compiler. Each run is wrapped within a 10-second timeout to detect hangs, ensuring the module can capture various types of runtime behaviors while avoiding indefinite stalls. This layered design provides a robust methodology for discovering cross-runtime semantic mismatches and exposing previously unknown bugs.

2) *Output Normalization.*: To minimize false positives and capture semantic output differences instead of just cosmetic divergences, we have developed a postprocessing layer which normalizes runtime outputs and maps diverse error messages to standardized categories. Each runtime (WASMTIME, WASMER, WAMR, and WASMEDGE) has a dedicated postprocessor function that detects recurring failure modes such as `out_of_bounds`, `integer_divide_by_zero`,

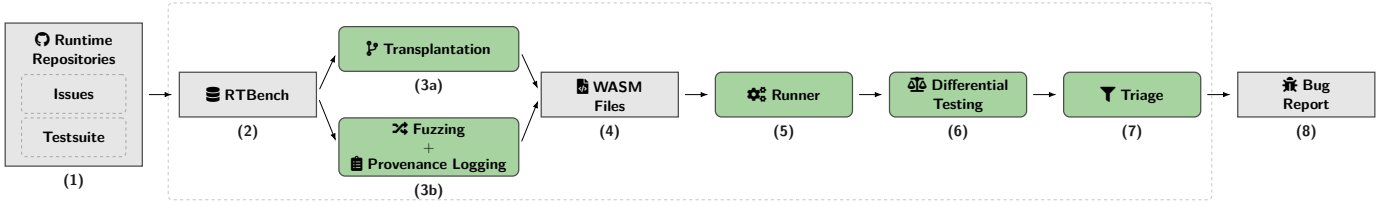


Fig. 2: Workflow of Study Design. Green rectangles represent processes, while grey rectangles denote data or artifacts.

TABLE II: Seed corpora used in our evaluation, their count, binary size, control-flow nesting depth, and baseline coverage achieved across four WebAssembly runtimes when replayed directly without fuzzing.

Seed Corpus	Count (K)	Size (KB)		Nesting Depth (K)		Branch Coverage (%) $\uparrow$			
		Mean	SD	Mean	SD	Wasmtime	Wasmer	WAMR	WasmEdge
LLVMBENCH	0.22	15	8.80	0.22	0.12	11.02	12.02	14.39	27.53
RTBENCH	0.58	584	3900	2.95	1.97	13.73	16.12	18.23	28.57
WASMBENCH	8.50	700	3200	1.86	1.48	12.31	11.97	17.71	27.67
SPECBENCH	200	0.27	0.13	0.00	0.00	10.89	10.01	15.32	19.81

`invalid_func_nargs`, or `unsupported`. Outputs that involve floating-point are normalized into consistent mantissa-exponent representations, and `+/- 'NaN'/'nan'` literals are standardized to ensure comparability. By filtering away runtime-specific warnings, trimming extraneous logs, and unifying divergent error messages, the module isolates semantically meaningful differences. This guarantees that any observed discrepancies across runtimes reflect true semantic divergences rather than cosmetic variations in error reporting.

D. Triageing

After identifying likely anomalies, we triage these cases by deduplication and simplification which are elaborated below.

1) *Deduplication.*: To efficiently triage recurring crashes across WebAssembly runtimes, we implemented a crash stack trace deduplication module that couples a Python callback in LLDB [27] with a Bash wrapper for automated execution. The wrapper script launches the program under LLDB in batch mode, installs a custom crash hook, and configures signal handling for `SIGSEGV` and `SIGABRT`. For Rust-based runtimes, we configured the panic mode to abort. When such abnormal terminations occur, the Python hook records the top five frames of the crashing thread, including function names and source file locations, filtered by a configurable path pattern. These frames are appended to a trace file and act as a concise fingerprint of the failure. By grouping test cases with identical top-frame sequences, the module coarsely clusters crashes that share the same underlying defect while avoiding over-fragmentation due to deeper stack variations. This design reduces redundant triage effort by collapsing large sets of failing test cases into representative clusters that can be more thoroughly inspected.

To triage differential outputs observed across runtimes, we built an execution tracing module that conditions on the divergent output itself. The module leverages LLDB to perform instruction-level tracing of a target program until a specified output string appears on `stdout`. When a mismatch is detected between runtimes, the expected differential output is supplied

as the tracing target, and execution halts as soon as this output is observed. At each step, the tracer records the current function, file, and line number, producing a detailed trace of the program’s execution path leading to the discrepancy. Since such traces can be verbose and include incidental logging, we focus on the most recent fifteen frames preceding the divergent output as a fingerprint for deduplication. By comparing these condensed trace signatures, the module distinguishes whether two test cases that generate the same output pattern actually expose the same underlying bug or separate issues. This approach improves the efficiency of triaging differential outputs.

2) *Test Reduction.*: Before reporting bugs, we apply automated test case reduction to minimize failing inputs while preserving the triggering behavior. This is handled through a reducer module built around the `wasm-tools shrink` utility, which iteratively simplifies WebAssembly binaries under the guidance of a custom predicate. The predicate runs the reduced module through our differential test engine and deduplication pipeline, compares the produced outputs against the reference failing outputs, and accepts the minimized candidate only if the failure is preserved. This ensures that irrelevant instructions and sections are pruned away, leaving behind the smallest input that still reproduces the crash or differential output. The module produces both a minimized `.wasm` file and a corresponding textual `.wat` representation for inspection.

By combining various deduplication strategies with a systematic reduction, we prepared the final list of bugs and test cases for reporting to the corresponding runtime issue trackers.

IV. OBJECTS AND QUESTIONS

We elaborate the objects we use in the study and the research questions we pose in the following.

A. Objects

Seed corpora. Table II provides a high-level comparison of the seed corpora used in our evaluation. We evaluate three input corpora in addition to our regression-based corpus

(RTBENCH): LLVMBENCH [6], WASMBENCH [28], [7], and SPECBENCH [5]. The other three corpora are publicly-available WASM datasets used in prior research.

In Table II, we show the mean testcase size and control-flow nesting depth as proxy metrics for testcase complexity. We note that SPECBENCH seeds have the lowest size and nesting depth because they are simple and focused seeds derived from the WASM specification [5]. In contrast, LLVMBENCH, WASMBENCH, and RTBENCH are relatively more complex and diverse corpora in terms of size and depth. RTBENCH has fewer seeds than most and they exhibit the highest standard deviation in both size and nesting depth, indicating that RTBENCH is highly diverse. Additionally, we observe that RTBENCH achieves higher initial coverage compared to the other corpora.

WASM runtimes. We target three runtimes: WASMER (20K★) [2], WASMEDGE (10K★) [4], and WAMR (6K★) [3], which are under active development and are highly popular. We also ran all the experiments on WASMTIME runtime, but since we did not find any bugs in it, we exclude it from the tables related to bug counts.

WASM fuzzers. We use 3 mutational WASM fuzzers: WADIFF [5], WASMAKER [7], and WAPPLIQUE [6]. We did not consider general-purpose fuzzers, as libFuzzer [29] or AFL++ [30], because prior work has shown that customized fuzzers are significantly more effective in the WebAssembly domain compared to general-purpose fuzzers [7], [6], [31]. We also have not considered purely generative fuzzers like WASMSMITH [26] since it does not require any seed corpora.

Following the standard for campaign-durations [32], [33], we ran each fuzzing campaign for 24 clock-hours. For a fair comparison, we ran each campaign with the same resources on an x86_64 AMD EPYC 9684X server to ensure that our cpu-hours would exceed that of the prior work on WASM fuzzing [6], [7], [5]. Moreover, to account for randomness, we repeated each campaign five times. We have used the `llvm-cov` tool to measure coverage across all the runtimes.

B. Research Questions

We pose the following research questions related to RTBENCH:

- 1) **RQ1: How effective is test transplantation for finding new bugs?**
- 2) **RQ2: Is RTBENCH a good seed corpus for fuzzing?**^{giorgi} **How does seed corpus choice affect fuzzing effectiveness across WASM runtimes?**^{giorgi}
- 3) **RQ3: How does transplantation compare to fuzzing?**
- 4) **RQ4: What are the characteristics of the bugs found?**
- 5) **RQ5: When do relevant seeds originate?**

The first question tests whether regression tests from one runtime can also find bugs in others, showing how reusable these tests are across runtimes. The second question checks how well fuzzing finds new bugs when using different seed corpora and how seed choice affects coverage and bug discovery. The third question compares transplantation and fuzzing with respect to bug-finding ability. The fourth question examines what kinds of bugs appear and what parts of the runtimes they affect. Finally, the fifth question looks at when

the effective seeds were created to learn whether older or newer tests are more useful for finding bugs today.

V. EVALUATION

To answer the research questions, we evaluate the bug counts and characterize a selected number of the bugs found. Additionally, we measure coverage and analyze the origination timeline of seeds.

A. RQ1: How effective is test transplantation for finding new bugs?

This question examines the baseline effects of RTBENCH—that is, the benefits of using RTBENCH directly on each runtime. We first analyze the impact of executing regression tests from one runtime on another, a process we refer to as “test transplantation” (§ V-A1). We then present a longitudinal study of the tests that revealed bugs (§ V-A2).

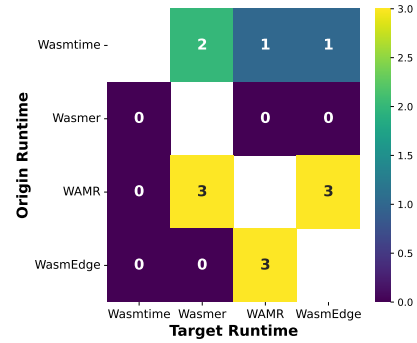


Fig. 3: Heatmap showing the number of bugs found from cross-runtime input transplantation using the RTBENCH seed corpora.

1) *RQ1.1: Can regression tests from one runtime directly reveal bugs in a different runtime?*: Figure 3 shows a heatmap matrix to illustrate the results of running regression test cases across runtimes. The matrix represents the effects of running regression test cases from an origin runtime (y-axis) in a target runtime (x-axis).

We observe that regression tests from all runtimes except WASMER, could reveal new bugs in the other runtimes, but the effect is uneven. For example, WAMR-derived seeds were particularly cross-fertile, triggering 3 bugs in WASMER and 3 bugs in WASMEDGE. This case is highlighted with 2 yellow-colored squares in the row labeled “WAMR”. Overall, these results indicate that test transplantation is capable of uncovering previously-hidden bugs in runtimes.

Figure 4 shows one such bug found in the WASMEDGE runtime by transplanting the test WAMR#3391. We have reduced the corresponding regression test for debugging and eventually reporting. Figure 4(a) shows the reduced code. The WAT code declares an `externref local` and immediately reads it with `local.get 0`, producing undefined data instead of a valid reference. During execution, the null value is interpreted, triggering a crash. To confirm whether this bug is indeed due to a lack of the necessary default initialization, we show a counterexample in Figure 4(b), where a similar

<pre> 1 (func (result externref) 2 (local externref) 3 local.get 0 4 5 6) </pre>	<pre> 1 (func (result externref) 2 (local externref) 3 ref.null extern 4 local.set 0 5 local.get 0 6) </pre>
(a) Bug-causing WAT code with no <code>externref</code> initialization.	(b) Corresponding non-crashing WAT code with explicit initialization.
<pre> 1 // WasmEdge/lib/executor/executor.cpp:164 2 auto *Inst = Val.get<RefVariant>().getPtr<Runtime::Instance 3 ::CompositeBase>(); 4 assuming(Inst); </pre>	
(c) Crash site in executor.cpp	

Fig. 4: Illustration of the **uninitialized externref bug** (WasmEdge#4325). The uninitialized local variable in (a) triggers a null pointer check in WASMEDGE (c). For comparison, the non-crashing WAT code is shown in (b). Confirmed and fixed with pull request (PR#4329).

WAT code that explicitly initializes `ref.null` does not trigger a crash. The WebAssembly Core Specification 4.2.1 requires that `externref` variables are initialized to `ref.null` [34]. Thus, the user is not responsible for explicitly initializing `externref` variables to `ref.null`. Hence, the absence of default initialization in WASMEDGE risked an eventual null pointer dereference. This bug was fixed by the pull request (PR#4329) shortly after reporting.

2) *RQ1.2: When do bugs found by transplantation originate in the buggy runtimes?*: Figure 5 shows the timelines of bugs revealed with transplantation as pairs of the form $\langle(\alpha, x, R_a), (\beta, x, R_b)\rangle$, where α denotes a point in time where a test case x appears in a bug report from runtime R_a and β denotes a point in time where x reveals a bug in runtime R_b . We report data quarterly to facilitate visualization. We use a binary search-like approach, similar to `git-bisect`, to identify the version of the buggy runtime that reveals the symptom for the first time.

The top-four horizontal lines from Figure 5 indicate the timelines associated with the origin of the test cases whereas the bottom-four horizontal lines indicate the timelines for revealing a bug with transplantation, i.e., the time a bug would be revealed for the first time in a different runtime. The boxes at both ends of each arrow show the ids of the test cases; the direction of the arrow (left or right) indicate if a bug existed in a runtime before it was reported in a different runtime. As we did not find any bugs on the WASMTIME runtime, the revelation bar of WASMTIME has been grayed out.

The visualization reveals two patterns. First, 7 of 13 bugs (i.e. 53.84%) existed long before their official reports (i.e., T1, T3, T7-10, and T13). Second, another 6 bugs (i.e. 46.15%) would only emerge in the observed runtimes long after they were originally reported elsewhere (e.g., T2, T4-6, T11-12). These patterns show that transplantation bugs exhibit substantial latency across runtimes: some bugs already exist, and other bugs will exist in the future. In both cases, transplantation of regression tests across runtimes is beneficial. The benefit

TABLE III: Number of unique bugs revealed by each combination of seed corpus, fuzzer, and runtime, aggregated over five 24-hour fuzzing campaigns. Column Σ aggregates unique bugs per corpus across fuzzers and runtimes. Bottom row aggregates per runtime across corpora and fuzzers. WASMTIME is excluded since no technique found a bug in it.

Seed Corpus	Fuzzer	WASMER	WAMR	WASMEDGE	Σ
LLVMBENCH	WADIFF	-	-	-	8
	WASMAKER	1	2	-	
	WAPPLIQUE	2	3	-	
WASMBENCH	WADIFF	-	-	-	5
	WASMAKER	2	1	-	
	WAPPLIQUE	-	1	1	
SPECBENCH	WADIFF	2	2	5	9
	WASMAKER	-	-	-	
	WAPPLIQUE	-	-	-	
RTBENCH	WADIFF	-	-	-	21
	WASMAKER	7	3	3	
	WAPPLIQUE	7	2	1	

would be immediate in the first case and would be delayed in the latter. Figure 5 also shows that most transplantation bugs cluster around 2022–2024, which was a period coinciding with major WebAssembly specification updates [35], [36].

Summary of RQ1. Regression test transplantation revealed a modest but important number of bugs in WASM runtimes: **13**, overall. For reference, the fuzzer WAPPLIQUE needed several hours of fuzzing to reveal 10 bugs in WASM runtimes.

B. RQ2: ~~Is RTBENCH a good seed corpus for fuzzing?~~^{giorgi} How does seed corpus choice affect fuzzing effectiveness across WASM runtimes?^{giorgi}

We compare four seed corpora across three fuzzers and three runtimes: three established baselines from prior WASM fuzzing work [6], [7], [5]: LLVMBENCH, WASMBENCH, and SPECBENCH. The fourth corpus is RTBENCH, our regression-derived corpus.^{giorgi}

Table III reports the unique bug counts for each corpus, fuzzer, and runtime combination. The three established corpora reveal 15 unique bugs combined. This is substantially lower than the numbers reported in their original publications. For example, WAPPLIQUE originally reported 20 bugs with LLVMBENCH [6], and WASMAKER originally reported 33 bugs with WASMBENCH [7]. This decline is consistent with the saturation effect discussed in Section I.^{giorgi}

In contrast, RTBENCH alone reveals 16 unique bugs. This is more than any individual established corpus, and also more than the three established corpora combined. Figure 6 shows this as a Venn diagram of bug overlap across corpora. Table III shows the breakdown of bug counts across all corpora, fuzzers, and runtimes. The same pattern holds across runtimes. Only WASMAKER and WAPPLIQUE were effective with RTBENCH, and both find a comparable number of bugs on all runtimes except WASMTIME. WADIFF did not yield bugs when paired

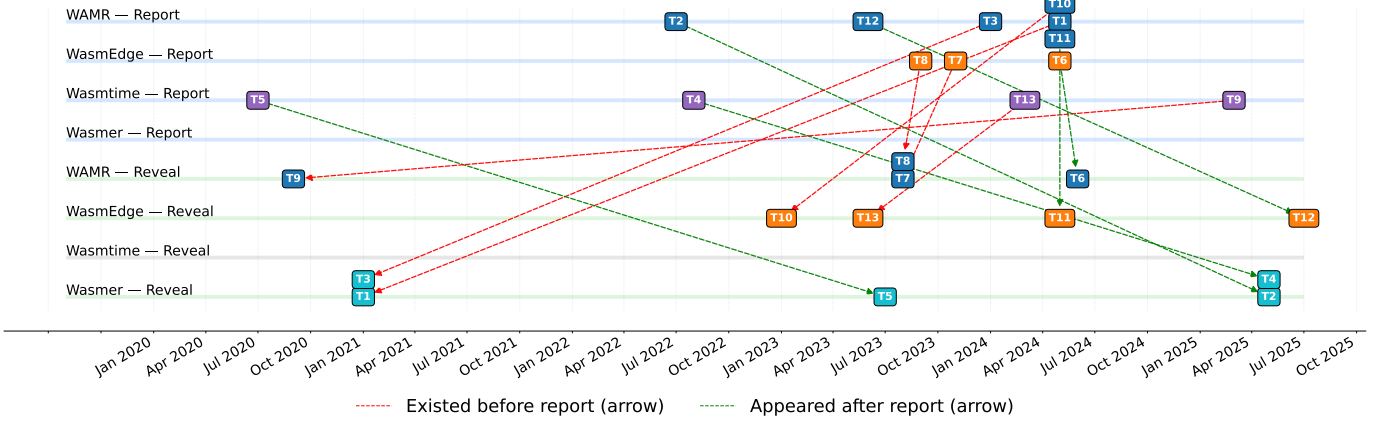


Fig. 5: Scatterplot showing timelines comparing dates of a buggy test case reported on one runtime and the revelation of another bug found with the same test case on another runtime. The small boxes show ids of specific test cases that were transplanted from one runtime to another. The directions of the arrows represent the runtime/timeline of a test case source and the corresponding runtime/timeline of the transplantation target.

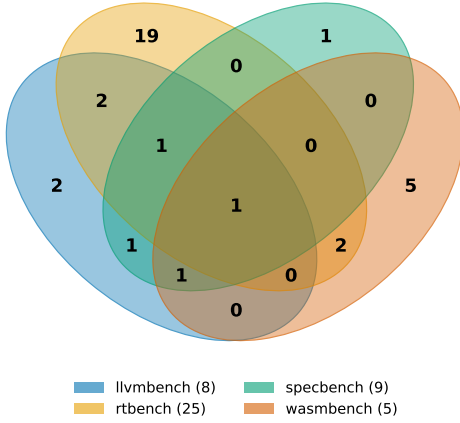


Fig. 6: Venn diagram showing fuzz bug membership of different seed corpora. **[[Ishrak: Update numbers. RTBENCH will be 21 instead of 25.]]****[[Ishrak: Intersection between llvmbench and rtbench will be 1 instead of 2.]]****[[Ishrak: Only rtbench will be 16 instead of 19.]]**

with RTBENCH. We discuss this fuzzer-corpus mismatch in Section VI-A.^{giorgi}

Figure 7 shows the coverage progress across all runtimes when using different fuzzers with all four seed corpora at 4 hours intervals. We observe that fuzzing improves coverage overall compared to the coverage achieved on the seed corpora prior to fuzzing (i.e. at timestep 0) by 5-10%. While WASMEDGE and WAMR runtimes achieved relatively higher coverage, WASMTIME had the lowest coverage for all seed corpora. We also note that for all runtimes, fuzzing with RTBENCH consistently achieved higher coverage compared to the other seed corpora. Also, most fuzzers plateau after around 12 hours, which aligns well with findings from prior work [37].

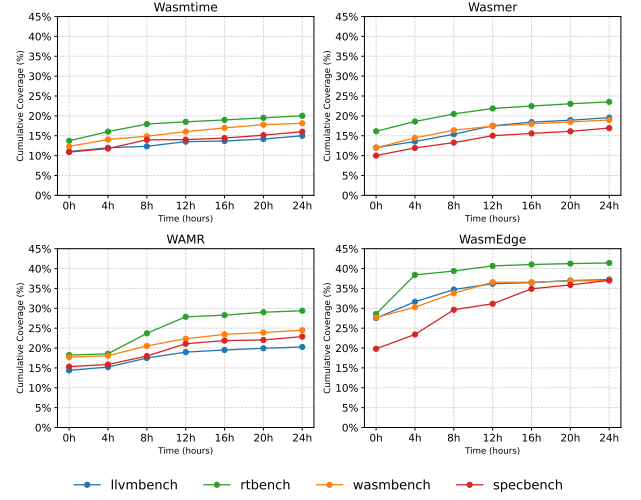


Fig. 7: Cumulative coverage progress when fuzzing with different seed corpora on all four runtimes. **[[Ishrak: Show variance with local pointwise boxplots. Duration will be reduced to 12h. Vertical spacing will be increased by increasing a per-subplot resolution. Each subplot will have their own vertical scaling.]]**

Summary of RQ2. Using RTBENCH as a seed corpus significantly improved fuzzing results. The fuzzers we analyzed revealed more bugs (21 **[[Ishrak: Use macro.]]** new bugs) and achieved higher coverage when using RTBENCH compared to other corpora.

C. RQ3: How does transplantation compare to fuzzing? **[[Ishrak: Reviewer: tautological?]]**

Table IV summarizes the status of bugs discovered by RTBENCH, divided into transplantation bugs, fuzz bugs, and their combined totals across four major WebAssembly runtimes: WASMTIME, WASMER, WAMR, and WASMEDGE. Several

TABLE IV: Status of reported bugs found using RTBENCH. The table shows transplantation bugs, fuzz bugs, and their totals. R=Reported, C=Confirmed, and F=Fixed.

Runtimes	Trans. Bugs			Fuzz Bugs			Total Bugs		
	R	C	F	R	C	F	R	C	F
WASMER	5	3	3	13	11	6	18	14	9
WAMR	4	3	0	4	3	0	8	6	0
WASMEDGE	4	4	3	4	3	2	8	7	5
Total	13	10	6	21	17	8	34	27	14

important observations emerge. First, fuzzing produced a substantially higher number of reported bugs compared to transplantation, indicating that fuzzing remains the more prolific bug-finding strategy overall. However, transplantation still uncovered unique bugs, particularly in WAMR and WASMEDGE. Second, WASMER had the largest number of total reported and confirmed bugs. Third, WASMEDGE and WASMER are the only runtimes with fixed bugs.

Another key observation is that the bugs discovered through transplantation and those uncovered by fuzzing effectively do not overlap. In cases where a fuzzing campaign appeared to expose the same bug as transplantation, we checked our fuzzing provenance logs to trace the original seeds that led to that bug-triggering test case [\[\[Ishrak: Reviewer asked if this is true by construction? i.e. through some filtering or deduplication? Answer: not really; when a transplanted seed directly reveals a new bug, we don't exclude it from fuzzing seed corpora. So, the same bug can also be found by a fuzzer. We attribute this bug to being found through direct transplantation and not to the fuzzer\]\]](#). We note that even when a bug initially seemed to be found by both transplantation and fuzzing, in all such cases, these bugs were attributable to transplantation alone. Hence, every one of the 13 bugs identified through transplantation was uniquely discovered by transplantation, with no additional contribution from any fuzzer.

Summary of RQ3. Fuzzing generally found more total bugs, but transplantation found different ones. None of the bugs from transplantation overlapped with those from fuzzing. Transplantation complements fuzzing.

D. RQ4: What are the characteristics of the bugs found?

Table V characterizes representative bugs revealed with RTBENCH as the seed corpus. These bugs are categorized by using terminology consistent with prior WASM testing work [6], [7]. We group them into two main categories: RUNTIME FAILURE and COMPILATION FAILURE. Runtime failures refer to crashes that occur during execution, which we further distinguish by the type of exception. A compilation failure occurs when a Wasm binary cannot be successfully compiled or instantiated, meaning it fails before execution begins.

We further categorize bugs based on their symptoms. Each testcase's output was analyzed to determine whether it belongs to the CRASH or OUTPUT-DIFFERENCE category. If a testcase triggers a runtime crash (e.g., SEGFALT or ABORTED), it is placed under the crash category; if it produces inconsistent

outputs across runtimes, it is classified as an output-difference bug. The categorization process begins by minimizing each testcase with `wasm - tools shrink` to its smallest reproducible form, then identifying the specific code segment responsible for the behavior. To validate our classification, we modified and relocated the identified code segments to confirm that they consistently reproduced the observed behavior. Finally, the structure of each minimized code pattern guided its placement into the most relevant category.

For instance, one representative runtime failure (Row 5) involves a recursive call inside a vector-returning function that never terminates and is eventually followed by an `unreachable` instruction, leading the runtime into an infinite loop before crashing. Another case (Row 13) represents a compilation failure where the compiler miscomputes lane extraction as an `i32` address and then incorrectly uses this address as the base for a SIMD load `v128.load8_splat`, triggering a validation error during instantiation.

The following examples illustrate two test cases that demonstrate a compilation failure and a runtime failure, respectively. Each case highlights the key instruction patterns and runtime behaviors that led to the observed failures.

```

1 (module...
2   (func (;0;) (type 0) (result i32 i32 v128)
3     i32.const 0
4     i32.const 0
5     v128.const i32x4 0x00000000 0x00000000 0
6       x00000000 0x00000000
7   )

```

Listing 1: (Compilation Failure Bug) (Wasmer#5667)

1) *Example 1:* reveals a crash pattern linked to incorrect handling of multi-value function returns involving SIMD vector types. The critical lines are `(type (func (result i32 i32 v128)))` and the instruction sequence `i32.const 0 → i32.const 0 → v128.const i32x4 0x00000000 0x00000000 0x00000000 0x00000000`. Here, the function is declared to return two `i32` values followed by a `v128` vector. While this is a valid multi-value return signature under the WebAssembly SIMD and multi-value proposals, certain runtimes—particularly LLVM-based JIT backends—mishandle the lowering of mixed scalar and SIMD return values. During compilation, the LLVM backend emits mismatched IR types between the declared function signature and the actual return instruction (e.g., returning `i64`, `i128` instead of the expected multi-value tuple), leading to an internal assertion failure: `Function return type does not match operand type of return inst.` This results in a compiler abort rather than a clean trap. Therefore, the failure is classified as a **Compilation Failure type-mismatch bug**, specifically triggered by the combination of multi-value returns and SIMD vector types that cause inconsistencies in function return type lowering during LLVM IR generation.

```

1 (module
2   (type (;0;) (func))...
3   (func (;0;)
4     br 0
5     ref.null 0
6     unreachable
7   )

```

TABLE V: Categories of a sample of bugs we report with RTBENCH.

ID	Category	Symptom	Found with	Error Conditions
1	Runtime Failure	Different Output	Transplantation	Operand of <code>ref.null</code> opcode is an id of a typed function reference.
2		Crash	Transplantation	Use of uninitialized reference type.
3		Different Output	Fuzzing	Generate a <code>v128</code> value from two constants, extract one signed lane as an <code>i32</code> , and use it as the base address in a memory load.
4		Different Output	Fuzzing	Value from <code>i16x8.extract_lane_s</code> is used as the address for <code>v128.load32x2_u</code> .
5		Crash	Fuzzing	Recursive call in a vector-returning function with no termination, followed by an unreachable.
6		Different Output	Fuzzing	Signed lane extraction • Using resulting <code>i32</code> value as the effective address.
7		Different Output	Fuzzing	Import and call the WASI function <code>fd_prestat_get</code> .
8	Compilation Failure	Crash	Fuzzing	Create a function with the return type (<code>i32</code> , <code>i32</code> , <code>v128</code>).
9		Crash	Fuzzing	NaN followed by <code>f32 SIMD</code> op followed by <code>f64 SIMD</code> op.
10		Different Output	Fuzzing	Non-zero SIMD constant followed by the SIMD <code>.ne</code> instruction followed by 4 different SIMD cast operations.
11		Crash	Fuzzing	Use <code>i32x4.lt_u</code> on a vector produced by a 64-bit compare (<code>f64x2.lt</code>).
12		Crash	Fuzzing	Lane extraction computes an <code>i32</code> address • Using the <code>i32</code> address as the base address for SIMD load <code>v128.load8_splat</code> .
13		Different Output	Transplantation	Include type constructor (<code>ref eq</code>) as a parameter type when defining a function type.
14		Different Output	Transplantation	Remove all functions.

8)

Listing 2: (Runtime Failure Category Bug) (WAMR#4480)

2) *Example 2*: demonstrates a runtime crash caused by incorrect handling of typed references within control-flow boundaries. The instruction sequence `br 0 → ref.null 0` is responsible for triggering the failure. The `br 0` instruction transfers control to the nearest enclosing block, rendering subsequent code unreachable, yet the engine must still correctly validate or lower it. Immediately following this, the instruction `ref.null 0` attempts to create a null reference of heap type index 0, which corresponds to the first function type defined in the module (`type (func)`). While this construct is syntactically valid under the reference-types proposal, certain runtimes—particularly JIT-based ones mishandle the combination of an unreachable code region with a typed reference operand, leading to inconsistent type-state tracking or invalid IR generation during lowering. This results in a runtime failure, specifically in the runtime’s implementation of reference-type validation and unreachable-block handling.

Summary of RQ4. With RTBENCH, we found a wide range of bugs, including compilation and runtime failures manifesting as crashes or output differences.

E. RQ5: When do relevant seeds originate?

Figure 8 highlights several important patterns in bug propagation from seeds to new test cases across different WebAssembly runtimes. First, seeds from WASMTIME and WAMR dominate the flow, producing a substantial number of downstream bugs, whereas WASMER and WASMEDGE appear more frequently as buggy runtimes than as primary seed sources. Second, 90% of the impactful seeds are concentrated between the 12-month and 36-month age bands, a period that coincides with major implementation upgrades from WebAssembly specification version 1 to 2 [35], [36]. 14.63% of the seeds from this period (12-36 months old) were useful for finding bugs. In contrast, very recent seeds that are

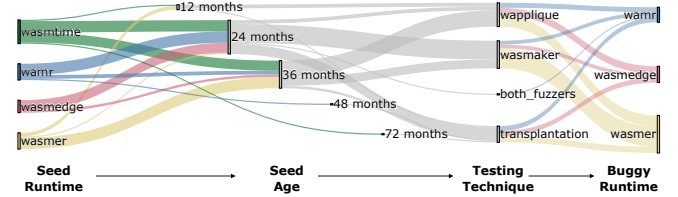


Fig. 8: Sankey diagram showing the lineage of bugs across WebAssembly runtimes. Flows connect the seed runtime, seed age, testing technique, and the buggy runtime where a new test case was observed. Source and target runtimes are color-coded consistently, while intermediate age and technique bands are shown in gray.

younger than 12 months, contribute less overall. Third, the oldest seeds (48 and 72 months) were only effective when used with transplantation, underscoring that transplantation provides a unique pathway to reactivating older seeds that fuzzing alone cannot exploit. Only 6.86% of the seeds that were either <12 months or >36 months old were useful in bug finding.

Summary of RQ5. 90% of the useful seeds came from regression tests that were 2–3 years old, aligning with the period when WebAssembly’s specification evolved from version 1 to 2. Older seeds, while less effective for fuzzing, still exposed new issues through transplantation.

VI. DISCUSSION

This section presents lessons learned and threats to validity.

A. Lessons Learned

We summarize key lessons learned as a set of observations and corresponding recommendations.

1. **Monitor all runtimes and integrate new regression tests.** Our study shows that regression tests from one runtime can directly reveal bugs in others without additional overhead and also serve as effective seeds for fuzzing

(§V-A & V-B). Moreover, 53.84% of the bugs found by transplantation existed long before in the observed runtimes (§V-A). So, we recommend the integration of regression tests from all runtimes as soon as they are updated.

2. **Prioritize regression tests created during major specification upgrades.** We observed that 90% of the useful transplanted tests and fuzzing seeds originated during major specification upgrades from version 1 to 2 (§V-A & V-E) [35], [36]. Specification upgrades are routine and version 3 has been recently finalized [38]. Therefore, as implementations for version 3 start rolling out, we recommend automated test generation from the specification updates similar to prior work on spec-based test generation [39].
3. **Continuously update and maintain RTBENCH.** The long-term utility of RTBENCH depends on both its evolution and its curation. As new regression tests are introduced to fix bugs or cover new specification features, they should be promptly incorporated into RTBENCH to capture up-to-date runtime behavior. We also found that older seeds were largely ineffective for both transplantation and fuzzing (§V-E). So, older seeds that exercise stable features or redundant paths should be periodically retired to maintain the efficiency of RTBENCH seed corpora [40], [41], [42].
4. **Adjust mutation strategy based on seed complexity.** Our results indicate that mutation effectiveness of WASM fuzzers is tied to seed complexity. We observed that WADIFF’s havoc mutations are effective on only SPECBENCH corpora (§V-B). It is worth noting that SPECBENCH contains simple and focused seeds (§IV-A). In contrast, WASMAKER and WAPPLIQUE’s splicing mutations perform well only when applied to complex and diverse seed corpora such as RTBENCH (§V-B, §IV-A). Based on our observations, we recommend a seed-complexity aware mutation strategy i.e. to apply havoc mutations on simple seeds and splicing on complex ones for WASM fuzzing.

B. Threats to Validity

Construct. A potential threat concerns how we define and count bugs. Some anomalies could stem from undefined runtime behaviors. Although our differential testing module normalizes outputs, nondeterministic effects may still yield spurious mismatches. To mitigate this, we identified patterns of known undefined behaviors and semi-automatically filtered such cases. **Internal.** Fuzzing inherently involves randomness, which can affect coverage and bug counts. To mitigate randomness bias, we executed each fuzzing campaign five times under fixed time budgets and identical configurations, then aggregated unique bug outcomes across runs. **External.** Another threat relates to the representativeness of our objects of study i.e. runtimes, fuzzers, and seed corpora. We focused on four mature and actively maintained runtimes and three widely used mutational fuzzers, which may not generalize to emerging engines or generative fuzzers. Similarly, the RTBENCH corpus was derived from regression tests of these four runtimes and may underrepresent other WebAssembly ecosystems. We mitigate this by selecting the most widely used runtimes, fuzzers, and seed corpora from prior work, and by releasing RTBENCH to enable extension in future studies.

VII. RELATED WORK

Regression Tests for Transplantation. ~~Test transplantation reuses existing tests across systems by adapting harnesses, configurations, and oracles. Prior studies demonstrate its effectiveness for JavaScript engines, databases, and mobile applications [43], [44], [45], [46], [47], [15], [48]. Prior work migrated regression tests between engines or platforms through lightweight adapters and standardized outputs. RTBENCH applies the same concept to WebAssembly by reusing regression tests from runtime repositories, confirming them on their original versions, and executing them across independent runtimes to detect divergences. This bridges the idea of test reuse across implementations while maintaining test intent.~~^{giorgi} Test transplantation reuses existing tests across systems by adapting harnesses, configurations, and oracles, and has proven effective for JavaScript engines, databases, and mobile applications [43], [44], [45], [46], [47], [15], [48]. In JavaScript engines, regression tests are transplanted across engines to find new bugs [43], and in database systems, a single test suite is reused across SQLite, PostgreSQL, and DuckDB [44]. RTBENCH applies this idea to WebAssembly by reusing regression tests from runtime repositories, confirming them on their original versions, and executing them across independent runtimes to detect divergences.^{giorgi}

Regression Tests for Fuzzing. ~~Regression tests can also serve as valuable fuzzing seeds. Mutational fuzzers such as WAPPLIQUE, WASMAKER, WADIFF, and others transform existing modules to explore differential or buggy runtime behavior [6], [7], [5], [49], [50], [31]. RTBENCH builds on these by deriving fuzzing seeds from real-world regression tests and bug reports, linking historical failures with systematic fuzzing and multi-runtime validation.~~^{giorgi} Regression tests can also serve as fuzzing seeds. Mutational fuzzers such as WAPPLIQUE, WASMAKER, WADIFF, and others [6], [7], [5], [49], [50], [31] transform existing modules to explore differential or buggy runtime behavior. RTBENCH builds on these by changing only the seed source, deriving seeds from real regression tests and bug reports. This exposes how much existing corpora have saturated: WAPPLIQUE and WASMAKER originally reported 20 and 33 bugs using LLVBENCH and WASMBENCH [6], [7], yet the same fuzzers on the same corpora now reveal far fewer new bugs, consistent with the saturation effect as runtimes mature [13], [14]. Using RTBENCH instead reveals the most new bugs of any corpus we evaluate.^{giorgi}

Differential Testing of WebAssembly Runtimes.^{giorgi} Differential testing runs the same module on several runtimes and treats output disagreements as bugs; WADIFF [5] and WASMAKER [7] follow this approach and serve as baseline fuzzers in our study. Two recent techniques add LLM assistance: LWDIFF [51] supports differential testing of WebAssembly runtimes, and DrWASI [52] targets WebAssembly System Interface (WASI) implementations. Both generate inputs with LLMs, whereas RTBENCH contributes real regression tests; the directions are complementary, and RTBENCH could seed an LLM-driven generator like LWDIFF.^{giorgi}

Other Related Works. Beyond regression-test reuse and fuzzing, several studies contribute to the broader understanding of WebAssembly reliability and fuzzing evaluation. Structure- and rule-guided Wasm generators such as WasmCFuzz, RGFuzz, and SwFuzz synthesize valid programs to exercise deep compiler and runtime behaviors [53], [54], [55]. Static and analysis-based approaches, including WasmChecker and WBSan, detect type and memory errors and undefined behaviors without relying on fuzzing [56], [57]. Formal verification efforts such as WasmRef-Isabelle provide a mechanized reference interpreter for validation and reasoning [58]. Broader surveys like the SoK “Analysis Techniques for WebAssembly” [59] and studies on Random Testing of WASI [60] contextualize the evolving Wasm testing landscape. Work on evaluation methodology also informs our design: comparative studies emphasize the importance of statistical rigor and reproducibility [61], [62], [63], while shared benchmarking infrastructures such as FuzzBench and GreenBench [32], [64] demonstrate reproducible experimental practices. Mutation-analysis research highlights the importance of separating coverage and mutation phases [65], and recent studies on mutator-level operator design [66] and multi-dimensional fuzzing metrics [33] offer guidance for controlled experimentation.^{giorgi} Other work targets WebAssembly reliability and fuzzing methodology more broadly. Structure and rule-guided generators such as WasmCFuzz, RGFuzz, and SwFuzz [53], [54], [55] build valid programs from a grammar or the specification. RTBENCH instead starts from real regression tests that already concentrate in bug-prone code, so the two are complementary as starting corpora. WasmChecker [56] detects type and memory errors through static analysis without fuzzing, while WBSan [57] provides sanitization to support binary-only fuzzing. WasmRef-Isabelle [58] provides a verified reference interpreter that can serve as a fuzzing oracle. The SoK on analysis techniques for WebAssembly [59] and work on random testing of WASI [60] survey this space. Studies on statistical rigor and reliable benchmarking [61], [62], [63] and platforms such as FuzzBench and GreenBench [32], [64] establish good practice for fuzzing evaluation. We follow this guidance by running repeated, fixed-budget campaigns and releasing our seeds. Mutation-analysis and mutator-level studies [65], [66] further motivate our use of default mutator settings across trials.^{giorgi}

VIII. CONCLUSIONS

This paper presents RTBENCH, a dataset of real-world WASM regression tests, and reports on an in-depth evaluation of two complementary strategies (namely, test transplantation and regression-seeded fuzzing) for uncovering bugs in WASM runtimes. Across three major runtimes, we discovered 34 bugs, with transplantation and fuzzing revealing distinct and non-overlapping issues. Our results show that regression tests remain valuable long after their creation to assist bug-fixes, especially those created during major specification updates. Overall, results indicate that the use of project metadata, such as issues and release notes, is a promising direction to improve effectiveness of WASM fuzzers.

DATA AVAILABILITY

Our results are publicly available at https://github.com/anonymous4submission2026/WASM_Fuzzing_Study.

REFERENCES

- [1] “GitHub - bytecodealliance/wasmtime: A lightweight WebAssembly runtime that is fast, secure, and standards-compliant — github.com,” <https://github.com/bytecodealliance/wasmtime>, [Accessed 01-01-2025].
- [2] “GitHub - wasmerio/wasmer: Fast, secure, lightweight containers based on WebAssembly — github.com,” <https://github.com/wasmerio/wasmer>, [Accessed 01-01-2025].
- [3] “GitHub - bytecodealliance/wasm-micro-runtime: WebAssembly Micro Runtime (WAMR) — github.com,” <https://github.com/bytecodealliance/wasm-micro-runtime>, [Accessed 01-01-2025].
- [4] “GitHub - WasmEdge/WasmEdge: WasmEdge is a lightweight, high-performance, and extensible WebAssembly runtime for cloud native, edge, and decentralized applications. It powers serverless apps, embedded functions, microservices, smart contracts, and IoT devices. — github.com,” <https://github.com/WasmEdge/WasmEdge>, [Accessed 01-01-2025].
- [5] S. Zhou, M. Jiang, W. Chen, H. Zhou, H. Wang, and X. Luo, “Wadiff: A differential testing framework for webassembly runtimes,” in *2023 38th IEEE/ACM International Conference on Automated Software Engineering (ASE)*. IEEE, 2023, pp. 939–950.
- [6] W. Zhao, R. Zeng, and Y. Zhou, “Waplique: Testing webassembly runtime via execution context-aware bytecode mutation,” in *Proceedings of the 33rd ACM SIGSOFT International Symposium on Software Testing and Analysis*, 2024, pp. 1035–1047.
- [7] S. Cao, N. He, X. She, Y. Zhang, M. Zhang, and H. Wang, “Wasmaker: Differential testing of webassembly runtimes via semantic-aware binary generation,” in *Proceedings of the 33rd ACM SIGSOFT International Symposium on Software Testing and Analysis*, 2024, pp. 1262–1273.
- [8] B. P. Miller, L. Fredriksen, and B. So, “An empirical study of the reliability of unix utilities,” *Communications of the ACM*, vol. 33, no. 12, pp. 32–44, 1990.
- [9] M. Woo, A. Orso, and T. Kim, “Krace: Data race detection for kernel applications,” in *Proceedings of the IEEE Symposium on Security and Privacy (SP)*. IEEE, 2013, pp. 596–610.
- [10] V.-T. Pham, M. Böhme, A. E. Santosa, D. Chollak, L. Tan, and A. Roychoudhury, “Smartseed: Smart seed generation for efficient fuzzing,” in *Proceedings of the 2016 11th Joint Meeting on Foundations of Software Engineering (ESEC/FSE)*. ACM, 2016, pp. 1–12.
- [11] M. Böhme, V.-T. Pham, and A. Roychoudhury, “Coverage-based grey-box fuzzing as markov chain,” *IEEE Transactions on Software Engineering*, vol. 45, no. 5, pp. 489–506, 2017.
- [12] V. Manès, H. Han, C. Han, S. K. Cha, M. Egele, E. J. Schwartz, and M. Woo, “The art, science, and engineering of fuzzing: A survey,” *IEEE Transactions on Software Engineering*, vol. 47, no. 11, pp. 2312–2331, 2021.
- [13] A. Groce and J. Regehr, “The Saturation Effect in Fuzzing &x2013; Embedded in Academia — [blog.regehr.org](http://blog.regehr.org/archives/1796),” <https://blog.regehr.org/archives/1796>, [Accessed 01-01-2025].
- [14] D. Liyanage, M. Böhme, C. Tantithamthavorn, and S. Lipp, “Reachable coverage: Estimating saturation in fuzzing,” in *2023 IEEE/ACM 45th International Conference on Software Engineering (ICSE)*. IEEE, 2023, pp. 371–383.
- [15] X. Yang, Y. Chen, E. Eide, J. Regehr, V. Le, M. Afshari, Z. Su, S. Kondratyev, E. Yermolovich *et al.*, “Finding and understanding bugs in c compilers (pldi 2011); compiler validation via equivalence modulo inputs (pldi 2014); yarpgen 2: Generating semantically well-defined programs for testing c/c++ compilers (cgo 2021),” 2011, consolidated entry for Csmith (PLDI 2011), EMI (PLDI 2014), and YARPGen 2 (CGO 2021).
- [16] C.-Y. Huang, C.-S. Kuo, and S.-P. Luan, “Evaluation and application of bounded generalized pareto analysis to fault distributions in open source software,” *IEEE Transactions on Reliability*, vol. 63, no. 1, pp. 309–319, 2013.
- [17] R. Yue, N. Meng, and Q. Wang, “A characterization study of repeated bug fixes,” in *2017 IEEE international conference on software maintenance and evolution (ICSME)*. IEEE, 2017, pp. 422–432.
- [18] L. Plein, W. C. Ouédraogo, J. Klein, and T. F. Bissyandé, “Automatic generation of test cases based on bug reports: a feasibility study with large language models,” in *Proceedings of the 2024 IEEE/ACM 46th International Conference on Software Engineering: Companion Proceedings*, 2024, pp. 360–361.

- [19] M. R. I. Rabin and M. A. Alipour, “Configuring test generators using bug reports: a case study of gcc compiler and csmith,” in *Proceedings of the 36th annual ACM symposium on applied computing*, 2021, pp. 1750–1758.
- [20] “Text Format &x2014; WebAssembly 3.0 (2025-10-16) — webassembly.github.io,” <https://webassembly.github.io/spec/core/text/index.html>, [Accessed 16-10-2025].
- [21] “wasm2wat(1) &x2014; Arch manual pages — man.archlinux.org,” <https://man.archlinux.org/man/wasm2wat.1.en>, [Accessed 21-10-2025].
- [22] “GitHub - bytecodealliance/wasm-tools: CLI and Rust libraries for low-level manipulation of WebAssembly modules — github.com,” <https://github.com/bytecodealliance/wasm-tools?tab=readme-ov-file>, [Accessed 21-10-2025].
- [23] Y. Zhang, M. Liu, H. Wang, Y. Ma, G. Huang, and X. Liu, “Research on webassembly runtimes: A survey,” *ACM Transactions on Software Engineering and Methodology*, 2024.
- [24] “Overview &x2014; WebAssembly 3.0 — webassembly.github.io,” <https://webassembly.github.io/spec/core/intro/overview.html#concepts>, [Accessed 08-10-2025].
- [25] “Validation Algorithm &x2014; WebAssembly 3.0 (2025-10-08) — webassembly.github.io,” <https://webassembly.github.io/spec/core/appendix/algorithm.html>, [Accessed 08-10-2025].
- [26] bytecodealliance, “wasm-smith: A webassembly test case generator.”
- [27] “&x1F41B; LLDB — lldb.llvm.org,” <https://lldb.llvm.org/>, [Accessed 23-10-2025].
- [28] A. Hilbig, D. Lehmann, and M. Pradel, “An empirical study of real-world webassembly binaries: Security, languages, use cases,” in *Proceedings of the web conference 2021*, 2021, pp. 2696–2708.
- [29] “libFuzzer: a library for coverage-guided fuzz testing,” <https://llvm.org/docs/LibFuzzer.html>, [Accessed 01-01-2025].
- [30] A. Fioraldi, D. Maier, H. Eißfeldt, and M. Heuse, “AFL++: Combining incremental steps of fuzzing research,” in *14th USENIX Workshop on Offensive Technologies (WOOT 20)*. USENIX Association, Aug. 2020.
- [31] L. Zhang, B. Zhao, J. Xu, P. Liu, Q. Xie, Y. Tian, J. Chen, and S. Ji, “Waltz: {WebAssembly} runtime fuzzing with {Stack-Invariant} transformation,” in *34th USENIX Security Symposium (USENIX Security 25)*, 2025, pp. 6159–6178.
- [32] J. Metzman, L. Szekeres, A. Arya *et al.*, “Fuzzbench: An open fuzzer benchmarking platform and service,” in *Proceedings of the ACM SIGSOFT International Symposium on Software Testing and Analysis (ISSTA)*, 2021.
- [33] Y. Li, S. Ji, Y. Chen, S. Liang, W. Lee *et al.*, “Unifuzz: A holistic and pragmatic metrics-driven platform for evaluating fuzzers,” in *Proceedings of the USENIX Security Symposium*, 2021.
- [34] “Runtime Structure &x2014; WebAssembly 3.0 (2025-10-08) — webassembly.github.io,” <https://webassembly.github.io/spec/core/exec/runtime.html#values>, [Accessed 01-10-2025].
- [35] “Change History &x2014; WebAssembly 3.0 (2025-10-08) — webassembly.github.io,” <https://webassembly.github.io/spec/core/appendix/changes.html>, [Accessed 10-10-2025].
- [36] “Wasm 2.0 Completed - WebAssembly — webassembly.org,” <https://webassembly.org/news/2025-03-20-wasm-2.0/>, [Accessed 03-25-2025].
- [37] W. Gao, V.-T. Pham, D. Liu, O. Chang, T. Murray, and B. I. Rubinstein, “Beyond the coverage plateau: A comprehensive study of fuzz blockers (registered report),” in *Proceedings of the 2nd International Fuzzing Workshop*, 2023, pp. 47–55.
- [38] “Wasm 3.0 Completed - WebAssembly — webassembly.org,” <https://webassembly.org/news/2025-09-17-wasm-3.0/>, [Accessed 22-10-2025].
- [39] N. Rao, E. Gilbert, H. Green, T. Ramananandro, N. Swamy, C. L. Goues, and S. Fakhoury, “Diffspec: Differential testing with llms using natural language specifications and code artifacts,” *arXiv preprint arXiv:2410.04249*, 2024.
- [40] Y. Li, Y. Zeng, X. Song, and S. Guo, “Improving seed quality with historical fuzzing results,” *Information and Software Technology*, vol. 179, p. 107651, 2025.
- [41] A. Rebert, S. K. Cha, T. Avgerinos, J. Foote, D. Warren, G. Grieco, and D. Brumley, “Optimizing seed selection for fuzzing,” in *23rd USENIX Security Symposium (USENIX Security 14)*, 2014, pp. 861–875.
- [42] A. Herrera, H. Gunadi, S. Magrath, M. Norrish, M. Payer, and A. L. Hosking, “Seed selection for successful fuzzing,” in *Proceedings of the 30th ACM SIGSOFT international symposium on software testing and analysis*, 2021, pp. 230–243.
- [43] I. Lima, J. Silva, B. Miranda, G. Pinto, and M. d’Amorim, “Exposing bugs in javascript engines through test transplantation and differential testing,” *Software Quality Journal*, 2021.
- [44] S. Zhong and M. Rigger, “Understanding and reusing test suites across database systems,” *Proceedings of the ACM on Management of Data*, 2025, sIGMOD.
- [45] X. Qin, H. Zhong, H. Yu *et al.*, “Testmig: Migrating gui test cases from ios to android,” in *Proceedings of the ACM SIGSOFT International Symposium on Software Testing and Analysis (ISSTA)*, 2019.
- [46] S. Talebipour, Y. Zhao, others., Y. Zhang, X. Zhu *et al.*, “Ui test migration across mobile platforms (mapit), ase 2021; and migratepro: Synthesis-based enhancement for gui test case migration, issta 2024,” 2021, consolidated entry for MAPIT (ASE 2021) and MigratePro (ISSTA 2024).
- [47] Y. Gao, W. Hu *et al.*, “Mut: Human-in-the-loop unit test migration,” in *Proceedings of the IEEE/ACM International Conference on Software Engineering (ICSE)*, 2024.
- [48] E. T. Barr, M. Harman, Y. Jia, A. Marginean, J. Petke, Y. Zhang, M. Kim, S. Soni, and M. D. Ernst, “Automated software transplantation (issta 2015); and automated transplantation and differential testing for clones (grafter) (icse 2017),” 2015, consolidated entry for ISSTA 2015 and ICSE 2017.
- [49] B. Jiang, Z. Li, Y. Huang, Z. Zhang, and W. Chan, “Wasmfuzzer: A fuzzer for webassembly virtual machines,” in *34th International Conference on Software Engineering and Knowledge Engineering, SEKE 2022*. KSI Research Inc., 2022, pp. 537–542.
- [50] P. Qian, X. Ying, J. Wang, L. Liu, L. Zhang, J. Chen, and Q. He, “Freewasm: Enhanced webassembly runtime fuzzing guided by parse tree mutation and snapshot,” *Proceedings of the ACM on Software Engineering*, vol. 2, no. ISSTA, pp. 159–181, 2025.
- [51] S. Zhou, J. Wang, H. Ye, H. Zhou, C. Le Goues, and X. Luo, “Lwdiff: An llm-assisted differential testing framework for webassembly runtimes,” in *Proceedings of the 47th IEEE/ACM International Conference on Software Engineering (ICSE)*, 2025.
- [52] Y. Zhang, N. He, J. Gao, S. Cao, K. Liu, H. Wang, Y. Ma, G. Huang, and X. Liu, “Drwasi: Llm-assisted differential testing for webassembly system interface implementations,” *ACM Transactions on Software Engineering and Methodology*, 2025.
- [53] X. Zhang, J. Wang, X. Du, and S. Liu, “Wasmcfuzz: Structure-aware fuzzing for wasm compilers,” in *Proceedings of the 2024 ACM/IEEE 4th International Workshop on Engineering and Cybersecurity of Critical Systems (EnCyCriS) and 2024 IEEE/ACM Second International Workshop on Software Vulnerability*, 2024, pp. 1–5.
- [54] J. Park, Y. Kim, and I. Yun, “Rgfuzz: Rule-guided fuzzer for webassembly runtimes,” in *2025 IEEE Symposium on Security and Privacy (SP)*. IEEE, 2025, pp. 920–938.
- [55] J. Wang, Z. Guo, X. Ying, P. Qian, and Y. Chen, “S w fuzz: Structure-sensitive webassembly fuzzing,” in *2024 31st Asia-Pacific Software Engineering Conference (APSEC)*. IEEE, 2024, pp. 343–352.
- [56] J. Zhuang, H. Jiang, and B. Hua, “Wasmchecker: Effectively detecting webassembly bugs via static program analysis,” in *2024 8th International Conference on Electrical, Mechanical and Computer Engineering (ICEMCE)*. IEEE, 2024, pp. 1669–1678.
- [57] X. Wu, J. He, L. Huang, C. Fu, and W. Wang, “Wbsan: Webassembly bug detection for sanitization and binary-only fuzzing,” in *Proceedings of the ACM on Web Conference 2025*, 2025, pp. 3311–3322.
- [58] C. Watt, M. Trela, P. Lammich, and F. Märkl, “Wasmref-isabelle: A verified monadic interpreter and industrial fuzzing oracle for webassembly,” *Proceedings of the ACM on Programming Languages*, vol. 7, no. PLDI, pp. 100–123, 2023.
- [59] H. Harnes and D. Morrison, “Sok: analysis techniques for webassembly,” *Future Internet*, vol. 16, no. 3, p. 84, 2024.
- [60] E. Stanley, “Random testing of the webassembly system interface,” 2024.
- [61] G. Klees, A. Ruef, B. Cooper, S. Wei, and M. Hicks, “Evaluating fuzz testing,” in *Proceedings of the ACM Conference on Computer and Communications Security (CCS)*, 2018.
- [62] A. Arcuri and L. Briand, “A practical guide for using statistical tests to assess randomized algorithms in software engineering,” in *Proceedings of the International Conference on Software Engineering (ICSE)*, 2011.
- [63] M. Böhme, V. Manès, Y. Chen, C. Yao, Y. Li, and S. Chattopadhyay, “On the reliability of coverage-based fuzzer benchmarking,” in *Proceedings of the International Conference on Software Engineering (ICSE)*, 2022.
- [64] J. Ounjai, M. Christakis, T. Chajed, P. Godefroid, A. Zeller, and M. Böhme, “Green fuzzer benchmarking,” in *Proceedings of the ACM SIGSOFT International Symposium on Software Testing and Analysis (ISSTA)*, 2023.
- [65] P. Görs, B. Mathis, E. Güler, K. Hassler, T. Holz, and A. Zeller, “Systematic assessment of fuzzers using mutation analysis,” in *Proceedings of the USENIX Security Symposium*, 2023.

- [66] J. Kukucka, L. Pina, P. Ammann, and J. Bell, “An empirical examination of fuzzer mutator performance,” in *Proceedings of the ACM SIGSOFT International Symposium on Software Testing and Analysis (ISSTA)*, 2024.